

AWS Basic Interview Q&A

04/26/2026 12:43:00

aws basic interview questions

Here are AWS basic interview questions (with clear answers)—perfect for 0–3 years...

Give me scenario-based AWS questions about real production issues

Here are real-world, scenario-based AWS interview questions—the kind interviewers ask ...

Give me AWS and Microservices architecture interview prep

Here's a complete AWS + Microservices Architecture interview prep guide—focused on...

Give me a hands-on project involving EC2, S3, RDS, and Load Balancer

Here's a hands-on, production-style AWS project using 👉 Amazon EC2 + Amazon S3 +...

Give me a live debugging scenario from this project

Here's a real, production-style live debugging scenario based on your project (Amazon EC...

give basic and advanced Q&A on (Amazon EC2 + Amazon S3 + Amazon RDS + Elastic Load...

Here's a complete interview prep (Basic → Advanced Q&A) for 👉 Amazon EC2 + Amazon...

Give me tricky cross-questions interviewers ask

Here are tricky cross-questions interviewers ask on 👉 Amazon EC2 + Amazon S3 +...

Give me debugging puzzles with multi-step failures

Here are real-world debugging puzzles with multi-step failures based on 👉 Amazon EC2 ...

lambda interview Q&A

Here's a complete AWS Lambda interview Q&A (Basic → Advanced → Scenario-based)—...

Thabira Meher_CV.pdf PDF according to my latest RCM project relate it to AWS services a...

Great—your resume gives strong context. You already worked on: RCM Healthcare doma...

Convert my experience into perfect interview answers

Got it—let's turn your experience into polished, confident, interview-ready answers you...

any basic questions missed on s3, ec2,aws lambda, aurora db, loadbalancer , ELB, Amazo...

You've already covered a lot, but yes—there are some important “basic-but-easily-missed...

advanced Q&A on s3, ec2,aws lambda, aurora db, loadbalancer , ELB, Amazon RDS

Here are advanced, interview-level Q&A on 👉 Amazon EC2 👉 Amazon S3 👉 AWS Lamb...

aws architecture explanation Explain the AWS Shared Responsibility Model. Difference...

Here's a clear, interview-ready AWS architecture explanation + Q&A for all your topics—...

Give me 50 most asked AWS interview questions company-wise

Here's a company-wise list of 50 MOST ASKED AWS interview questions (based on pattern...

answer all the 50 questions

That's a big set—but exactly what you need. I'll give you concise, interview-ready answers...

Ask deep follow-ups like an Amazon interviewer and answer the question also

Alright—now we'll step into Amazon-style deep follow-ups. These are not surface...

Interrupt me with cross-questions like a real interviewer

Alright—let's do this like a real Amazon-style interview. I'll ask → you answer → I interrupt...



Here are **AWS basic interview questions (with clear answers)**—perfect for 0–3 years experience and quick revision before interviews.

1. What is AWS?

Answer:

Amazon Web Services (AWS) is a cloud platform that provides on-demand computing services like servers, storage, databases, networking, and more on a pay-as-you-go basis.

2. What are the main AWS service categories?

Answer:

- Compute → Amazon EC2
 - Storage → Amazon S3
 - Database → Amazon RDS
 - Networking → Amazon VPC
 - Security → AWS IAM
-

3. What is EC2?

Answer:

Amazon EC2 is a virtual server in the cloud that allows you to run applications just like a physical machine.

4. What is S3?

Answer:

Amazon S3 is used to store files (images, videos, backups). It offers high durability (99.999999999%).

5. What is RDS?

Answer:

Amazon RDS is a managed database service that supports MySQL, PostgreSQL, Oracle, etc.

 6. What is IAM?

Answer:

AWS IAM controls access to AWS services using users, roles, and policies.

 7. What is VPC?

Answer:

Amazon VPC is a private network in AWS where you launch resources securely.

 8. What is Auto Scaling?

Answer:

Automatically increases or decreases EC2 instances based on traffic/load.

 9. What is Elastic Load Balancer?

Answer:

Distributes incoming traffic across multiple EC2 instances to ensure high availability.

 10. What is EBS?

Answer:

Amazon EBS is block storage used with EC2 (like a hard disk).

 11. What is AWS Region and Availability Zone?

Answer:

- Region → Geographical area (e.g., Mumbai)
 - AZ → Isolated data centers within a region
-

 12. What is a Security Group?

Answer:

Acts as a firewall for EC2 instances controlling inbound/outbound traffic.

13. What is Route 53?

Answer:

Amazon Route 53 is a DNS service used to route traffic to applications.

14. What is AWS pricing model?

Answer:

- Pay-as-you-go
 - Reserved instances
 - Spot instances
-

15. Difference between S3 and EBS?

Answer:

Feature	S3	EBS
Type	Object Storage	Block Storage
Usage	Files	Disk for EC2
Access	HTTP	Attached to EC2

16. What is IAM Role vs IAM User?

Answer:

- User → Individual access
 - Role → Temporary access (used by services like EC2)
-

17. What is Lambda?

Answer:

AWS Lambda lets you run code without managing servers.

18. What is CloudWatch?

Answer:

Amazon CloudWatch monitors logs, metrics, and alarms.

19. What is CloudFormation?

Answer:

Used to create infrastructure using code (Infrastructure as Code).

20. What is Elastic IP?

Answer:

A static public IP address for EC2.

Bonus: Real Interview Rapid Fire

Q: How do you secure EC2?

→ Security Groups + IAM roles + Key pairs

Q: How do you store large files?

→ S3

Q: How do you scale automatically?

→ Auto Scaling + Load Balancer

Q: How do you monitor apps?

→ CloudWatch

If you want next level:

-  ↳ Scenario-based AWS questions (real production issues)
-  ↳ AWS + Microservices architecture interview prep
-  ↳ Hands-on project (EC2 + S3 + RDS + Load Balancer)

Just tell me 

4/26/2026, 9:00:57 AM



Give me scenario-based AWS questions about real production issues

4/26/2026, 9:01:43 AM



Here are **real-world, scenario-based AWS interview questions**—the kind interviewers ask to test *practical production experience*, not theory. I'll include **what interviewer expects + how you should answer**.



1. Website Suddenly Down (High Traffic Spike)

? Scenario

Your application hosted on Amazon EC2 becomes unreachable during peak traffic.

✅ What interviewer expects

- Check CPU, memory via Amazon CloudWatch
- Verify Load Balancer health
- Check Auto Scaling status

💡 Ideal Answer

- Check EC2 health + CloudWatch metrics
- Verify Elastic Load Balancing health checks
- Scale instances using Auto Scaling
- If not configured → immediate fix + RCA

👉 **Key point:** Always mention **Auto Scaling + Load Balancer**



2. Application is Slow

? Scenario

Users complain that API response time increased.

✅ Expected Approach

- Check DB latency
- Check EC2 CPU
- Analyze logs

💡 Answer

- Monitor using CloudWatch
- Check DB in Amazon RDS
- Enable caching (Amazon ElastiCache)
- Scale vertically or horizontally

👉 Bonus: mention **query optimization**

🔒 3. Unauthorized Access Detected

? Scenario

Suspicious login activity detected in AWS account.

💡 Answer

- Rotate credentials
- Check AWS IAM policies
- Enable MFA
- Audit logs via AWS CloudTrail

👉 Key: **Security + auditing mindset**

💰 4. AWS Bill Suddenly Increased

? Scenario

Billing spiked unexpectedly.

💡 Answer

- Check billing dashboard
- Identify unused EC2/S3
- Look for data transfer spikes
- Use cost tools

👉 Mention:

AWS Cost Explorer

👉 Bonus: "We set billing alerts to avoid future issues"

5. Data Loss in S3

? Scenario

Important files deleted from Amazon S3

💡 Answer

- Check versioning
- Restore previous versions
- Enable lifecycle policies

👉 Key: Always say

"Enable versioning for recovery"

6. Cannot Access EC2 Instance

? Scenario

SSH to EC2 not working.

💡 Answer

- Check Security Group (port 22 open?)
 - Check key pair
 - Check instance status
 - Check subnet routing in Amazon VPC
-

7. Deployment Failed in Production

? Scenario

New release breaks application.

💡 Answer

- Rollback deployment
- Use blue-green deployment
- Check logs

👉 Mention:

AWS CodeDeploy

⚡ 8. High Latency for Global Users

? Scenario

Users from other countries experience slow response.

💡 Answer

- Use CDN
 - 👉 Amazon CloudFront
 - Use multiple regions
-

🔥 9. Database Connection Timeout

? Scenario

App cannot connect to DB.

💡 Answer

- Check DB status in RDS
 - Check security groups
 - Check connection pool limits
-

🧩 10. Microservices Communication Failure

? Scenario

Services cannot talk to each other.

💡 Answer

- Check service discovery

- Check networking (VPC, subnets)
- Use messaging

👉 Mention:

Amazon SQS or

Amazon SNS

🧠 11. Logs Not Available for Debugging

? Scenario

Production issue but no logs.

💡 Answer

- Enable centralized logging
 - Use CloudWatch Logs
-

⚙️ 12. CI/CD Pipeline Failure

? Scenario

Build pipeline fails.

💡 Answer

- Check pipeline logs
- Verify permissions
- Fix environment config

👉 Mention:

AWS CodePipeline

🎯 Pro Interview Tip (VERY IMPORTANT)

In **every answer**, follow this structure:

👉 1. Detect (monitoring)

👉 2. Diagnose (logs + metrics)

👉 3. Fix (scaling/config/security)

👉 4. Prevent (alerts/automation)

🔥 If you want next level

I can give you:

- 💣 ↪️ **Tricky counter-questions interviewers ask**
- 🏗️ ↪️ **Full AWS architecture (EC2 + ALB + RDS + S3 + CloudFront)**
- 🐞 ↪️ **Live debugging simulation (step-by-step commands)**

Just tell me 👍

4/26/2026, 9:01:43 AM



Give me AWS and Microservices architecture interview prep

4/26/2026, 9:02:10 AM



Here's a **complete AWS + Microservices Architecture interview prep guide**—focused on *real-world architecture, design thinking, and what interviewers actually expect*.

1. Standard AWS Microservices Architecture (Production)

◆ High-Level Flow

Client → CloudFront → API Gateway → Microservices → DB/Cache → Messaging

◆ Services Used

- CDN → Amazon CloudFront
- API Layer → Amazon API Gateway
- Compute → Amazon EC2 / AWS Lambda
- Containers → Amazon ECS / Amazon EKS
- Database → Amazon RDS / Amazon DynamoDB
- Cache → Amazon ElastiCache
- Messaging → Amazon SQS / Amazon SNS
- Monitoring → Amazon CloudWatch

2. Real Microservices Architecture (Explain in Interview)

Example: E-commerce System

Services:

- User Service
- Order Service
- Payment Service
- Inventory Service

Flow:

1. Client hits API Gateway
 2. Request routed to respective service
 3. Services communicate:
 - Sync → REST
 - Async → SQS/SNS
-

3. Synchronous vs Asynchronous Communication

◆ Synchronous (REST)

- Direct call between services
- Faster response
- Tight coupling

👉 Example: Order → Payment API call

◆ Asynchronous (Event-driven)

- Uses queues/events
- Loose coupling
- More scalable

👉 Example:

- Order placed → Event → Inventory updates
-

4. Important Architecture Patterns

◆ API Gateway Pattern

- Single entry point
- Handles auth, routing

👉 Use: Amazon API Gateway

◆ Database per Service

- Each microservice has its own DB

- Avoid shared DB
-

◆ Circuit Breaker Pattern

- Prevent cascading failures

👉 Example:

- Payment service down → fallback response
-

◆ Saga Pattern (VERY IMPORTANT 🔥)

Used for distributed transactions

Flow:

1. Order created
2. Payment processed
3. Inventory updated

If failure → rollback previous steps

🔒 5. Security in Microservices

Must Mention:

- IAM roles → AWS IAM
 - JWT authentication
 - API Gateway authorizers
 - HTTPS everywhere
-

📊 6. Observability (INTERVIEW FAVORITE)

Tools:

- Logs → CloudWatch Logs
- Metrics → CloudWatch
- Tracing → AWS X-Ray

👉 Say:

“We use centralized logging and distributed tracing to debug issues”

7. CI/CD Pipeline Architecture

Flow:

Code → Build → Test → Deploy

Tools:

- AWS CodePipeline
- AWS CodeBuild
- AWS CodeDeploy

8. Deployment Strategies (VERY IMPORTANT)

Types:

- Blue-Green Deployment
- Canary Deployment
- Rolling Deployment

👉 Example:

Use CodeDeploy for blue-green

9. Scaling Strategy

Horizontal Scaling:

- Auto Scaling + Load Balancer

👉 Elastic Load Balancing

10. Common Production Problems (Must Know)

? Service Down

→ Use Auto Scaling + health checks

? DB Bottleneck

→ Add caching (ElastiCache)

? High Latency

→ Use CloudFront + multi-region

? Failure Handling

→ Retry + circuit breaker

11. How to Answer Architecture Questions (Golden Template)

When interviewer asks:

👉 “Design scalable system”

Answer like this:

1. Entry point → API Gateway
2. Services → Microservices (loosely coupled)
3. Communication → REST + SQS
4. DB → RDS/DynamoDB
5. Caching → ElastiCache
6. Scaling → Auto Scaling + Load Balancer
7. Monitoring → CloudWatch + X-Ray
8. Security → IAM + JWT

12. Tricky Interview Questions

? Why not monolith?

→ Hard to scale, tightly coupled

? How do you handle failures?

→ Retry + circuit breaker + fallback

? How do services find each other?

→ Service discovery (ECS/EKS or internal DNS)

? How do you maintain data consistency?

→ Saga pattern

🧠 13. Advanced Points (To Impress Interviewer)

- Event-driven architecture (SQS + SNS)
 - Idempotency in APIs
 - Distributed tracing (X-Ray)
 - Rate limiting (API Gateway)
 - Caching strategies
-

🚀 If you want next level (highly recommended)

I can give you:

- 🧩 ↳ Full system design (like Flipkart / Uber) on AWS
- 💻 ↳ Spring Boot microservices + AWS deployment code
- 🛠 ↳ Live debugging scenario (step-by-step like production)
- 🎯 ↳ Mock interview with cross-questions

Just tell me what you want next 👍

4/26/2026, 9:02:10 AM



Give me a hands-on project involving EC2, S3, RDS, and Load Balancer

4/26/2026, 9:02:44 AM



Here's a **hands-on, production-style AWS project** using

👉 Amazon EC2 + Amazon S3 + Amazon RDS + Elastic Load Balancing

This is exactly the kind of project you can explain in interviews.



Project: Scalable Web App (Spring Boot + AWS)



Use Case

Build a **User Management App**:

- Users register/login
- Upload profile image
- Data stored in DB
- Images stored in S3
- App scaled across multiple servers



Architecture

```
User → Load Balancer → EC2 (App instances)
                        → RDS (Database)
                        → S3 (Images)
```



Step 1: Create RDS Database

Use:

👉 Amazon RDS

Steps:

1. Choose MySQL/PostgreSQL

2. Set DB name: `userdb`
 3. Create username/password
 4. Enable public access (for testing)
 5. Note endpoint
-

Step 2: Create Spring Boot App

Features:

- Register user
- Fetch users
- Upload image

application.yml

```
spring:
  datasource:
    url: jdbc:mysql://<RDS-ENDPOINT>:3306/userdb
    username: admin
    password: password
  jpa:
    hibernate:
      ddl-auto: update
```

Entity

```
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private String email;
    private String imageUrl;
}
```

Controller

```

@RestController
@RequestMapping("/users")
public class UserController {

    @PostMapping
    public User create(@RequestBody User user) {
        return repo.save(user);
    }

    @GetMapping
    public List<User> getAll() {
        return repo.findAll();
    }
}

```

Step 3: Store Images in S3

Use:

👉 Amazon S3

Steps:

1. Create bucket
2. Enable public access (or IAM-based access)

Upload Code

```

@Autowired
private AmazonS3 s3Client;

public String uploadFile(MultipartFile file) {
    String fileName = System.currentTimeMillis() + "_" + file.getOriginalFilename();

    s3Client.putObject("your-bucket", fileName, file.getInputStream(), new
ObjectMetadata());

    return s3Client.getUrl("your-bucket", fileName).toString();
}

```

Step 4: Launch EC2 Instances

Use:

👉 Amazon EC2

Steps:

1. Launch 2 instances (Ubuntu)
2. Install Java

```
sudo apt update  
sudo apt install openjdk-17-jdk -y
```

3. Deploy your JAR

```
java -jar app.jar
```

Step 5: Configure Load Balancer

Use:

👉 Elastic Load Balancing

Steps:

1. Create Application Load Balancer
2. Add both EC2 instances
3. Configure health check (/users)
4. Use DNS of Load Balancer

Step 6: Security Setup

- EC2 → allow port 8080
- RDS → allow EC2 access
- IAM role → allow S3 access

👉 Use:

AWS IAM

Step 7: Monitoring

👉 Use:

Amazon CloudWatch

- Monitor CPU
- Set alarms

Step 8: Test Flow

1. Hit Load Balancer URL
2. Create user → stored in RDS
3. Upload image → stored in S3
4. Load Balancer distributes traffic

What You Say in Interview

👉 "I built a scalable microservice deployed on EC2 behind a Load Balancer, used RDS for structured data, and S3 for file storage. The system supports horizontal scaling and high availability."

Real Enhancements (Say This to Impress)

- Add Auto Scaling
- Use CloudFront for CDN
- Add caching (ElastiCache)
- Use CI/CD (CodePipeline)
- Dockerize app

Want next level?

I can give you:

-  ↪ Docker + Kubernetes version of this project
-  ↪ Terraform script (Infra as Code)
-  ↪ Resume-ready explanation
-  ↪ Live debugging scenario from this project

Just tell me 👍

4/26/2026, 9:02:44 AM



Give me a live debugging scenario from this project

4/26/2026, 9:03:15 AM



Here's a **real, production-style live debugging scenario** based on your project (Amazon EC2 + Amazon S3 + Amazon RDS + Elastic Load Balancing).

This is exactly how you should explain in interviews 📌



Scenario: Application Down (503 from Load Balancer)

? Problem

Users hit Load Balancer URL → **503 Service Unavailable**



Step-by-Step Debugging (What interviewer expects)



Step 1: Check Load Balancer Health

👉 Go to:

Elastic Load Balancing

- Check **Target Group** → **Health Status**
- ❌ Instances showing **Unhealthy**



Conclusion:

Problem is NOT LB → problem is inside EC2 or app



Step 2: Check EC2 Instance Health

👉 Open:

Amazon EC2

- Instance status: Running ✅
- System status: OK ✅

👉 SSH into instance:

```
ssh -i key.pem ubuntu@<ip>
```



Step 3: Check Application Running?

```
ps -ef | grep java
```

Issue Found:

App NOT running

Step 4: Start Application

```
java -jar app.jar
```

Error:

```
Cannot connect to database
```

Step 5: Debug Database Issue

 Go to:

Amazon RDS

Check:

- DB status → Available 
 - Endpoint correct?
 - Security group?
-

Root Cause Found:

RDS security group **NOT** allowing EC2 access

Fix

 Update RDS Security Group:

- Allow inbound MySQL (3306)
 - Source → EC2 Security Group
-

Step 6: Restart App

```
java -jar app.jar
```

Now app starts successfully 

Step 7: Verify Load Balancer

- Target group → Healthy 
 - Access URL → Working 
-

Final Root Cause (Say this clearly)

👉 “The issue was due to misconfigured security group where EC2 could not connect to RDS. After fixing inbound rules, application recovered.”

Interviewer Follow-up Questions (VERY IMPORTANT)

How would you prevent this?

👉 Answer:

- Use proper security group templates
- Add health checks
- Use monitoring

👉 Mention:

Amazon CloudWatch

What if app was running but still unhealthy?

👉 Answer:

- Check health check endpoint `/users`
 - Check port mismatch
 - Check logs
-

? What logs will you check?

```
tail -f app.log
```

💣 Advanced Twist (Real Production)

🔥 Situation:

Only 1 instance unhealthy

👉 Possible reasons:

- Deployment failed
- Port conflict
- Memory issue

🧠 Golden Debugging Approach (Say This Always)

1. Check Load Balancer
2. Check EC2
3. Check Application
4. Check Database
5. Fix & verify

🚀 Want next level?

I can simulate:

- 💣 ↳ High CPU issue debugging
- 💣 ↳ S3 upload failure scenario
- 💣 ↳ Auto Scaling not working
- 💣 ↳ Microservices communication failure

Just tell me 👍

4/26/2026, 9:03:15 AM



give basic and advanced Q&A on
(Amazon EC2 + Amazon S3 + Amazon RDS + Elastic Load Balancing).

4/26/2026, 9:03:44 AM



Here's a **complete interview prep (Basic → Advanced Q&A)** for
👉 Amazon EC2 + Amazon S3 + Amazon RDS + Elastic Load Balancing

This is structured exactly how interviewers expect 📌

🟢 BASIC Q&A

💻 EC2 (Compute)

? What is EC2?

Answer:

Amazon EC2 is a virtual server where you can deploy and run applications.

? What is AMI?

Answer:

Amazon Machine Image → Template used to launch EC2 instances.

? What is a Security Group?

Answer:

Acts as a firewall controlling inbound/outbound traffic.

? What is Key Pair?

Answer:

Used to securely connect (SSH) to EC2.

📦 S3 (Storage)

? What is S3?

Answer:

Amazon S3 stores files (images, videos, backups) with high durability.

? What is a Bucket?

Answer:

Container to store objects in S3.

? What is Versioning?

Answer:

Keeps multiple versions of a file for recovery.

 RDS (Database)

? What is RDS?

Answer:

Amazon RDS is a managed relational database service.

? What DB engines are supported?

MySQL, PostgreSQL, Oracle, SQL Server

? What is Multi-AZ?

Answer:

Automatic failover to standby DB in another AZ.

 Load Balancer

? What is ELB?

Answer:

Elastic Load Balancing distributes incoming traffic across multiple servers.

? Types of Load Balancer?

- Application Load Balancer (ALB)
 - Network Load Balancer (NLB)
 - Classic Load Balancer
-

INTERMEDIATE Q&A

EC2

? Difference between vertical and horizontal scaling?

- Vertical → Increase instance size
 - Horizontal → Add more instances
-

? What is Auto Scaling?

Answer:

Automatically adjusts EC2 instances based on traffic.

S3

? Difference between S3 and EBS?

S3	EBS
Object storage	Block storage
Accessible via HTTP	Attached to EC2

? What is S3 lifecycle policy?

Answer:

Automatically moves data to cheaper storage (Glacier).

RDS

? What is Read Replica?

Answer:

Read-only copy of DB to improve performance.

? Difference: Multi-AZ vs Read Replica?

Multi-AZ	Read Replica
High availability	Performance
Sync replication	Async replication

Load Balancer

? What is Health Check?

Answer:

Checks if instances are healthy before routing traffic.

? Sticky Sessions?

Answer:

Routes same user to same instance.

ADVANCED Q&A (REAL INTERVIEW LEVEL)

EC2

? How do you debug EC2 not reachable?

Answer:

- Check Security Group
- Check NACL
- Check instance status
- Check SSH key

? How do you secure EC2?

Answer:

- Use IAM roles
- Restrict ports
- Disable root login

S3

? How do you secure S3 bucket?

👉 Answer:

- Block public access
 - Use IAM policies
 - Enable encryption
-

? What is pre-signed URL?

👉 Answer:

Temporary URL to access private S3 object.

RDS

? How do you handle DB failure?

👉 Answer:

- Use Multi-AZ
 - Automatic failover
-

? How do you optimize DB performance?

👉 Answer:

- Indexing
 - Query optimization
 - Read replicas
 - Caching
-

Load Balancer

? How does ALB work internally?

👉 Answer:

- Receives request
 - Checks health
 - Routes to healthy instance
-

? What happens if all instances fail?

👉 Answer:

- LB returns 5xx error
 - Need Auto Scaling / fallback
-

💣 SCENARIO-BASED QUESTIONS (MUST PREPARE)

? EC2 CPU is 100%

👉 Answer:

- Check processes
 - Scale using Auto Scaling
 - Optimize app
-

? S3 file not accessible

👉 Answer:

- Check bucket policy
 - Check permissions
-

? RDS connection timeout

👉 Answer:

- Check security group
 - Check DB load
-

? Load Balancer showing unhealthy instances

👉 Answer:

- Check health check path
 - Check app running
 - Check port mismatch
-

🎯 FINAL INTERVIEW TIP

Whenever asked anything, answer like this:

- 👉 **1. What it is**
 - 👉 **2. Why used**
 - 👉 **3. Real example**
-

If you want next level

I can give you:

- ↳ 🎯 Mock interview (I ask, you answer)
- ↳ 🧠 Tricky cross-questions interviewers ask
- ↳ 🏗️ Full system design using these services
- ↳ 💻 Hands-on coding + deployment walkthrough

Just tell me 👍

4/26/2026, 9:03:44 AM



Give me tricky cross-questions interviewers ask

4/26/2026, 9:04:14 AM



Here are **tricky cross-questions interviewers ask** on

👉 Amazon EC2 + Amazon S3 + Amazon RDS + Elastic Load Balancing

These are designed to **test depth, not definitions**. I'll show:

👉 Question → ❌ Weak answer → ✅ Strong answer

💣 1. "Why not store everything in S3 instead of RDS?"

❌ Weak:

"S3 is for files, RDS is for database"

✅ Strong:

"S3 is object storage and doesn't support relational queries, joins, or transactions. For structured data and ACID compliance, I use RDS. S3 is ideal for unstructured data like images."

👉 Interviewer checks: **Do you understand data modeling**

💣 2. "If one EC2 fails, will Load Balancer handle everything automatically?"

❌ Weak:

"Yes, it will handle"

✅ Strong:

"Load Balancer routes traffic only to healthy instances, but we must configure health checks and Auto Scaling. Without Auto Scaling, capacity reduces and can impact performance."

👉 Checks: **High availability understanding**

💣 3. "Why not connect frontend directly to EC2 instead of Load Balancer?"

❌ Weak:

"No idea"

✅ Strong:

"Load Balancer provides fault tolerance, distributes traffic, and avoids single point of failure. Direct EC2

access risks downtime and scalability issues.”

4. “What happens if your RDS goes down?”

 Weak:

“Application will fail”

 Strong:

“With Multi-AZ enabled in RDS, failover happens automatically to standby instance. Also, retry logic and connection pooling should be implemented in the app.”

 Checks: **Resilience thinking**

5. “Why not store images in EC2 instead of S3?”

 Weak:

“Not recommended”

 Strong:

“EC2 storage is limited and not durable. S3 provides high durability, scalability, and easy access via URLs. Also, EC2 replacement would lead to data loss.”

6. “How does Load Balancer know instance is unhealthy?”

 Weak:

“It checks”

 Strong:

“Through health check endpoints (like `/users`). If the response fails or times out, the instance is marked unhealthy and traffic is stopped.”

7. “What if all EC2 instances are healthy but app still fails?”

 Weak:

“Restart app”

 Strong:

“Possible causes:

- DB connection issue (RDS)
- External dependency failure
- Incorrect health check endpoint

I would check logs and metrics using monitoring.”

👉 Mention:

Amazon CloudWatch

💣 8. "How do you handle sudden traffic spike?"

❌ Weak:

"Add more servers"

✅ Strong:

"I use Auto Scaling with Load Balancer. Also optimize using caching and CDN if needed."

💣 9. "Can S3 replace a database?"

❌ Weak:

"Yes"

✅ Strong:

"No. S3 lacks querying, indexing, and transactions. It's storage, not a database."

💣 10. "How do you secure communication between EC2 and RDS?"

❌ Weak:

"Using password"

✅ Strong:

"Using Security Groups (allow only EC2 SG to access RDS), IAM roles if needed, and optionally SSL connections."

💣 11. "Why is your system still slow even after adding more EC2 instances?"

❌ Weak:

"No idea"

✅ Strong:

"Possible bottlenecks:

- Database (RDS)
 - Network latency
 - Lack of caching
- Scaling compute alone doesn't solve DB bottlenecks."

💣 12. "What happens if S3 is public?"

❌ Weak:

"Anyone can access"

✅ Strong:

"Yes, which is a security risk. We should block public access and use IAM policies or pre-signed URLs."

💣 13. "How do you handle file upload failure in S3?"

❌ Weak:

"Retry"

✅ Strong:

"Implement retry with exponential backoff, validate upload success, and log failures. Also ensure correct IAM permissions."

💣 14. "How do you debug 503 error from Load Balancer?"

❌ Weak:

"Restart everything"

✅ Strong:

"I check:

1. Target group health
 2. EC2 instance status
 3. Application logs
 4. DB connectivity (RDS)"
-

💣 15. "Why do we need Auto Scaling if Load Balancer exists?"

❌ Weak:

"Both are same"

✅ Strong:

"Load Balancer distributes traffic; Auto Scaling adjusts capacity. They work together for scalability."

🎯 Golden Strategy (VERY IMPORTANT)

Whenever interviewer cross-questions:

👉 Always include:

- Scaling
 - Security
 - Failure handling
 - Monitoring
-

🔥 Want to go even deeper?

I can:

- 🎯 ↳ Take a **mock interview** (I'll grill you like real interviewer)
- 💣 ↳ Give **system design questions with counter-attacks**
- 🐛 ↳ Give **debugging puzzles (multi-step failures)**

Just say: *"Start mock interview"* 👍

4/26/2026, 9:04:14 AM



Give me debugging puzzles with multi-step failures

4/26/2026, 9:04:41 AM



Here are **real-world debugging puzzles with multi-step failures** based on

👉 Amazon EC2 + Amazon S3 + Amazon RDS + Elastic Load Balancing

Each puzzle simulates **actual production incidents** where **multiple things break at once** (this is what senior-level interviews test).

💣 PUZZLE 1: “Everything Looks Healthy, But App Fails”

? Symptoms

- Load Balancer → Healthy ✅
- EC2 → Running ✅
- App → Returning **500 error**

🧠 Hidden Issues (multi-step)

1. App is running on **port 8081**
2. Load Balancer health check is hitting **port 8080**
3. Health check endpoint returns OK (dummy endpoint), but real APIs fail

✅ Expected Debugging Flow

- Check LB → looks fine
- SSH EC2 → check running port

```
netstat -tulnp
```

- Compare with LB config

🎯 Final Answer

👉 Port mismatch + incorrect health check endpoint

PUZZLE 2: “Intermittent Failures (Works Sometimes)”

Symptoms

- Sometimes API works
 - Sometimes 503 error
-

Hidden Issues

1. 2 EC2 instances behind Elastic Load Balancing
 2. One instance:
 - App running 
 - Health check still passing (wrong endpoint)
-

Debugging Flow

- Check target group → both healthy
 - Hit instance individually → one failing
-

Final Answer

 Health check misconfigured → unhealthy instance still receiving traffic

PUZZLE 3: “App Down After Deployment”

Symptoms

- After deployment → app crashes
 - Load Balancer shows unhealthy
-

Hidden Issues

1. New version deployed
 2. DB schema changed
 3. Amazon RDS not updated
-

✅ Debugging Flow

- Check logs:

```
tail -f app.log
```

- Error: `Column not found`
-

🎯 Final Answer

👉 DB schema mismatch (deployment issue)

💣 PUZZLE 4: “Image Upload Failing Only in Production”

? Symptoms

- Local → works
 - Production → upload fails
-

🧠 Hidden Issues

1. Missing IAM role on EC2
 2. No permission to access Amazon S3
-

✅ Debugging Flow

- Check error: `Access Denied`
 - Verify IAM role
-

🎯 Final Answer

👉 Missing S3 permissions

💣 PUZZLE 5: “High Latency Even After Scaling”

? Symptoms

- Added more EC2 instances

- Still slow
-

Hidden Issues

1. DB (RDS) CPU at 95%
 2. No caching
 3. All instances hitting DB
-

Debugging Flow

- Check metrics in:
 - 👉 Amazon CloudWatch
-

Final Answer

- 👉 Database bottleneck (not compute issue)
-
-

PUZZLE 6: “503 Error Only for Some Users”

Symptoms

- Some users see error
 - Others don't
-

Hidden Issues

1. Sticky sessions enabled
 2. One EC2 instance broken
 3. Users stuck to that instance
-

Final Answer

- 👉 Sticky session causing routing to bad instance
-
-

PUZZLE 7: “EC2 Running But Not Reachable”

? Symptoms

- Instance is running
 - Cannot access app
-

🧠 Hidden Issues

1. Security Group → port not open
 2. OR app bound to `localhost`
-

🎯 Final Answer

👉 Network or binding issue

💣 PUZZLE 8: "Data Not Saving But No Error"

? Symptoms

- API returns success
 - Data not in DB
-

🧠 Hidden Issues

1. Wrong DB endpoint configured
 2. Writing to different DB
-

🎯 Final Answer

👉 Misconfigured RDS endpoint

💣 PUZZLE 9: "S3 Files Missing Randomly"

? Symptoms

- Files disappear
-

🧠 Hidden Issues

1. Lifecycle policy enabled
 2. Auto-delete after X days
-

Final Answer

👉 Misconfigured lifecycle rule

PUZZLE 10: "System Works, Then Suddenly Crashes"

? Symptoms

- Works fine initially
 - Crashes after some time
-

Hidden Issues

1. Memory leak in app
 2. EC2 memory full
-

Debugging

top

Final Answer

👉 Memory exhaustion issue

HOW TO ANSWER THESE IN INTERVIEW

Always follow:

- 👉 1. Check Load Balancer
- 👉 2. Check EC2
- 👉 3. Check Application
- 👉 4. Check Database
- 👉 5. Check External services (S3, etc.)

🔥 Want the ultimate prep?

I can:

- 🎯 ↳ Conduct **live mock debugging interview**
- 💣 ↳ Ask you puzzles → you answer → I correct
- 🧠 ↳ Give **system design failure scenarios (Uber/Netflix level)**

Just say: **"Start debugging mock interview"** 👍

4/26/2026, 9:04:42 AM



Here's a **complete AWS Lambda interview Q&A (Basic → Advanced → Scenario-based)**—focused on what interviewers actually test.

⚡ BASIC Q&A

? What is AWS Lambda?

Answer:

AWS Lambda is a serverless compute service that runs code without managing servers. You only pay for execution time.

? What is serverless?

Answer:

No need to manage infrastructure (servers). AWS handles scaling, patching, and availability.

? What languages are supported?

- Java
 - Python
 - Node.js
 - Go
 - .NET
-

? What is a Lambda trigger?

Answer:

An event that invokes Lambda.

👉 Examples:

- Amazon S3 (file upload)
- Amazon API Gateway (HTTP request)
- Amazon SQS (queue message)

? What is cold start?

Answer:

Delay when Lambda runs for the first time or after inactivity.

🟡 INTERMEDIATE Q&A

? How does Lambda scale?

Answer:

Automatically creates multiple instances to handle concurrent requests.

? What is execution timeout?

Answer:

Max time a Lambda can run (up to 15 minutes).

? What is memory configuration?

Answer:

You allocate memory (128MB–10GB), which also affects CPU power.

? What is environment variable?

Answer:

Used to store config (DB URL, API keys).

? What is IAM role in Lambda?

Answer:

Permissions assigned to Lambda to access AWS services.

👉 Example:

Access S3 using:

AWS IAM

🔴 ADVANCED Q&A

? How do you reduce cold start?

✓ Answer:

- Use provisioned concurrency
 - Keep function warm
 - Reduce package size
-

? What is concurrency in Lambda?

✓ Answer:

Number of parallel executions Lambda can handle.

? Difference: Lambda vs EC2?

Lambda	EC2
Serverless	Managed server
Auto scaling	Manual scaling
Pay per execution	Pay per uptime

? How do you handle large file processing?

✓ Answer:

- Use S3 trigger
 - Break into chunks
 - Use step functions if needed
-

? What is Lambda Layer?

✓ Answer:

Reusable code (libraries, dependencies)

? How do you debug Lambda?

✓ Answer:

- Use logs in
👉 Amazon CloudWatch
- Add structured logging

SCENARIO-BASED QUESTIONS

? Lambda timing out

✓ Answer:

- Increase timeout
- Optimize code
- Move heavy tasks async

? Lambda not triggered from S3

✓ Answer:

- Check event config
- Check permissions
- Verify bucket notification

? High cost in Lambda

✓ Answer:

- Reduce execution time
- Optimize memory
- Avoid unnecessary invocations

? Duplicate processing from SQS

✓ Answer:

- Use idempotency
- Handle retries properly

TRICKY CROSS-QUESTIONS

? Can Lambda replace EC2?

✗ Weak: Yes

✓ Strong:

“No. Lambda is ideal for event-driven, short-lived tasks. For long-running processes or full control, EC2

is better.”

? Why is your Lambda slow even with low traffic?

✓ Answer:

- Cold start issue
 - External dependency delay
-

? Can Lambda connect to RDS?

✓ Answer:

Yes, but:

- Use VPC config
- Manage connection pooling carefully

👉 DB:

Amazon RDS

? What happens if Lambda fails?

✓ Answer:

- Retry (async)
 - Send to DLQ (SQS/SNS)
-

🧠 REAL ARCHITECTURE EXAMPLE

Flow:

User → API Gateway → Lambda → RDS
→ S3

🎯 FINAL INTERVIEW TIP

When answering Lambda questions, always include:

- 👉 Event-driven
 - 👉 Scaling
 - 👉 Cost optimization
 - 👉 Failure handling
-

If you want next level

I can give you:

-  ↳ Full Lambda project (Java + API Gateway + S3 trigger)
-  ↳ Debugging scenarios (real production failures)
-  ↳ Mock interview (Lambda-focused)
-  ↳ Comparison: Lambda vs Kubernetes vs EC2

Just tell me 👍

4/26/2026, 9:05:19 AM



Thabira Meher_CV.pdf
PDF

according to my latest RCM project relate it to AWS services and give basic and advanced Q&A if missed

4/26/2026, 9:11:21 AM



Great—your resume gives strong context. You already worked on:

- **RCM Healthcare domain (Availity – Essential Pro)**
- Microservices (Spring Boot + Kafka)
- AWS: **S3 + Aurora (RDS)** + EC2 (implicitly via deployment)
- Docker + Kubernetes

Now I'll **map your real project** → **AWS architecture** and give **interview-ready Q&A (basic → advanced → tricky)**.



1. Your RCM Project → AWS Mapping



Real Flow (based on your experience)

```
Frontend (React)
    ↓
API Gateway / Load Balancer
    ↓
Microservices (Spring Boot on EC2/K8s)
    ↓
Aurora DB (RDS)
    ↓
S3 (documents, reports)
    ↓
Kafka (async processing)
```



AWS Mapping

- Compute → Amazon EC2
- Storage (files, EOBs, claims docs) → Amazon S3

- Database (claims, patients, billing) → Amazon RDS (Aurora)
 - Traffic distribution → Elastic Load Balancing
 - Async events → Kafka (you already used)
-

BASIC Q&A (RCM CONTEXT)

? How did you use S3 in your RCM project?

✓ Answer:

"We used Amazon S3 to store claim documents, reports, and file uploads. It provided scalable and durable storage, and we stored only file references in DB."

? Why Aurora (RDS) for RCM?

✓ Answer:

"We used Amazon RDS (Aurora) because RCM requires transactional consistency (claims, payments), and Aurora provides high availability and performance."

? Where did EC2 fit in your project?

✓ Answer:

"Our Spring Boot microservices were deployed on Amazon EC2 instances (or containers running on EC2), enabling scalable backend processing."

? Why Load Balancer?

✓ Answer:

"We used Elastic Load Balancing to distribute traffic across microservices, ensuring high availability and fault tolerance."

INTERMEDIATE Q&A (REAL EXPERIENCE LEVEL)

? How did you handle large file uploads (EOB/claims)?

✓ Answer:

- Upload to S3 directly
 - Store URL in DB
 - Avoid storing binary in RDS
-

? How did microservices communicate?

✅ Answer:

- Sync → REST APIs
 - Async → Kafka (event-driven for claim processing)
-

? How did you ensure DB performance?

✅ Answer:

- Used Aurora (RDS)
 - Query optimization
 - Connection pooling
 - Read replicas (if needed)
-

? How did you scale your services?

✅ Answer:

- Horizontal scaling via EC2 instances
 - Load Balancer distributes traffic
-

🔴 ADVANCED Q&A (INTERVIEW GOLD 🔥)

? In RCM, how do you ensure data consistency across microservices?

✅ Answer:

"We used event-driven architecture with Kafka and followed eventual consistency patterns. For critical flows (like claim submission), we ensured transactional integrity at service level."

? Why not store documents in DB instead of S3?

✅ Answer:

"Storing large files in DB affects performance and increases cost. S3 is optimized for object storage and provides better scalability and durability."

? How do you handle failure in claim processing?

✅ Answer:

- Retry mechanism (Kafka consumer)
- Dead-letter queue

- Logging and monitoring
-

? How do you secure patient data (VERY IMPORTANT)?

✅ Answer:

- IAM roles
 - Encryption (S3 + RDS)
 - Restricted access via security groups
 - HTTPS communication
-

? What happens if RDS becomes slow?

✅ Answer:

- Identify slow queries
 - Add indexing
 - Scale DB
 - Introduce caching
-

💣 TRICKY CROSS-QUESTIONS (RCM + AWS)

? Why is your system slow even after scaling EC2?

✅ Strong Answer:

“Scaling compute doesn’t help if DB (RDS) is bottleneck. In RCM, heavy queries on claims can slow system, so DB optimization or caching is required.”

? How do you handle duplicate claim processing?

✅ Answer:

- Idempotent APIs
 - Unique transaction IDs
 - Kafka message deduplication
-

? What if S3 upload succeeds but DB save fails?

✅ Answer:

- Use transactional handling

- Cleanup orphan files
 - Or use async confirmation flow
-

? What if one microservice fails in claim workflow?

✅ Answer:

"Use Saga pattern to rollback or compensate previous steps."

? Why not use only REST instead of Kafka?

✅ Answer:

"REST creates tight coupling. Kafka enables asynchronous processing, improves scalability, and handles high-volume claim processing efficiently."

💣 REAL DEBUGGING QUESTION (YOU WILL GET THIS)

? Claim submission failing randomly. What will you check?

✅ Answer:

1. Load Balancer health
 2. EC2 logs
 3. Kafka queue lag
 4. RDS performance
 5. S3 upload issues
-

🎯 HOW TO ANSWER IN INTERVIEW (VERY IMPORTANT)

Always connect:

- 👉 Business (RCM)
 - 👉 Technology (AWS)
 - 👉 Problem (scaling / reliability)
-

🚀 FINAL STRONG ANSWER (Use this line 🔥)



"In my RCM project, we built microservices on EC2 behind a Load Balancer, used Aurora (RDS) for transactional healthcare data, S3 for document storage, and Kafka for asynchronous processing. This architecture ensured scalability, fault tolerance, and high performance."

🔥 Want next level (highly recommended)

I can:

- 🎯 ↳ Convert your experience into **perfect interview answers (tell me about your project)**
- 💣 ↳ Ask **mock interviewer questions specifically on your resume**
- 🧠 ↳ Give **system design (RCM system on AWS)**

Just tell me 👍

4/26/2026, 9:11:22 AM



Convert my experience into perfect interview answers

4/26/2026, 9:41:37 AM



Got it—let's turn your experience into **polished, confident, interview-ready answers** you can deliver smoothly.

I'll give you **exact answers to speak**, not theory.

1. "Tell me about your current project" (MOST IMPORTANT)

Writing



I'm currently working as a Senior Software Engineer at Availity on a healthcare RCM product called Essential Pro.

The system is built using a microservices architecture where we use Spring Boot for backend services and React with TypeScript for the frontend. Each service is responsible for a specific domain like claims processing, analytics, and user management.

From a cloud perspective, we use AWS services where our applications are deployed on Amazon EC2, traffic is routed through a Load Balancer for high availability, and we use Amazon RDS (Aurora) for storing transactional healthcare data like claims and payments.

For document storage, such as claim files and reports, we use Amazon S3 and store only metadata or file references in the database.

We also use Kafka for asynchronous communication between services, especially for processing claims and analytics workflows, which helps us decouple services and handle high volumes efficiently.

My role involves designing and developing microservices, integrating REST APIs, handling production issues, and ensuring performance and scalability of the system.

2. "Explain your architecture"

Writing



Our architecture follows a standard microservices pattern.

Client requests from the frontend are routed through a Load Balancer, which distributes traffic across multiple backend services running on EC2 instances.

Each microservice is independently deployed and communicates either synchronously via REST APIs or asynchronously using Kafka.

We use Amazon RDS (Aurora) for structured data like claims, billing, and patient details, ensuring transactional consistency.

For file storage, we use Amazon S3 to handle large documents like EOBs and reports.

This setup allows us to scale services independently, maintain high availability, and handle large volumes of healthcare transactions efficiently.

3. "How do you handle scalability?"

Writing



We handle scalability at multiple levels.

At the compute layer, we scale horizontally by adding more EC2 instances, and the Load Balancer distributes traffic across them.

At the application level, we use microservices, so each service can scale independently based on load.

For heavy processing tasks like claim workflows, we use Kafka to process events asynchronously, which prevents bottlenecks.

At the database level, we optimize queries and can use read replicas in RDS if needed.

Overall, this combination ensures the system can handle increasing traffic without performance degradation.

4. "How do you ensure security (especially healthcare data)?"

Writing



Since we deal with healthcare data, security is critical.

We use IAM roles to control access to AWS resources, ensuring services only have the permissions they need.

Data stored in Amazon S3 and RDS is encrypted, and all communication between services happens over HTTPS.

We also restrict access using security groups so that only specific services can communicate with the database.

Additionally, we follow best practices like not exposing sensitive data in logs and securing APIs using authentication mechanisms.

5. "Tell me about a production issue you handled"

Writing



One production issue I handled was when our application started returning 503 errors through the Load Balancer.

Initially, the infrastructure looked fine, but after debugging, I found that the application was unable to connect to the database.

The root cause was a misconfigured security group where the EC2 instances were not allowed to access the RDS instance.

After updating the inbound rules and restarting the application, the system recovered.

From this, we implemented better monitoring and alerts using CloudWatch to detect such issues early.

6. "Why did you choose S3 instead of DB for files?"

Writing



We chose Amazon S3 for storing files because it is designed for scalable and durable object storage. Storing large files in a relational database like RDS would impact performance and increase storage costs.

Instead, we store files in S3 and keep only the file URL or metadata in the database, which improves performance and keeps the system efficient.

7. "How do microservices communicate?"

Writing



We use both synchronous and asynchronous communication.

For real-time operations, we use REST APIs between microservices.

For background processing, such as claim processing and analytics, we use Kafka for event-driven communication.

This helps us decouple services and improve scalability and fault tolerance.

8. "How do you ensure system reliability?"

Writing



We ensure reliability through multiple strategies.

We use a Load Balancer to distribute traffic and avoid single points of failure.

Our services are deployed across multiple instances, so if one instance fails, others can handle the traffic.

We also use proper logging and monitoring to quickly identify and resolve issues.

Additionally, we design services to handle failures gracefully, using retries and fallback mechanisms where needed.

9. "What challenges did you face?"

Writing



One major challenge was handling high-volume claim processing without impacting system performance.

Initially, synchronous processing caused delays, so we redesigned the flow using Kafka for asynchronous processing.

This significantly improved performance and allowed the system to handle larger volumes efficiently.

Another challenge was debugging distributed systems issues, which we addressed by improving logging and monitoring.

10. Strong Closing Line (Use Anywhere)

Writing



Overall, my experience involves building scalable microservices on AWS using EC2, RDS, and S3, combined with event-driven architecture using Kafka, ensuring high performance, reliability, and secure handling of healthcare data.

If you want to go next level

I can:

-  ↳ Turn this into a **2-minute + 5-minute + deep-dive version**
-  ↳ Do a **live mock interview on your resume**
-  ↳ Add **Spring Boot + AWS code-based questions**

Just tell me 👍

4/26/2026, 9:41:42 AM



any basic questions missed on s3, ec2,aws lambda, aurora db, loadbalancer , ELB, Amazon RDS cover it.

4/26/2026, 12:35:41 PM



You've already covered a lot, but yes—there are **some important “basic-but-easily-missed” questions** that interviewers love to ask to catch gaps. I'll cover them cleanly across:

- 👉 Amazon EC2
- 👉 Amazon S3
- 👉 AWS Lambda
- 👉 Amazon RDS (Aurora included)
- 👉 Elastic Load Balancing



EC2 – MISSED BASIC QUESTIONS

? What is instance type?

Answer:

Defines CPU, memory, storage capacity (e.g., t2.micro, m5.large).

? What is EBS vs Instance Store?

- EBS → Persistent storage
- Instance Store → Temporary (lost on stop/terminate)

? What happens when you stop vs terminate EC2?

Action	Result
Stop	Instance paused, EBS retained
Terminate	Instance deleted permanently

? What is Elastic IP?

Answer:

Static public IP attached to EC2.

? What is user-data script?

Answer:

Script that runs at instance startup (used for automation).

S3 – MISSED BASIC QUESTIONS

? What is an object in S3?

Answer:

File + metadata stored in a bucket.

? What are storage classes?

- Standard
 - IA (Infrequent Access)
 - Glacier
-

? What is S3 durability?

Answer:

99.999999999% (11 9's durability)

? Difference: public vs private bucket?

- Public → accessible via URL
 - Private → restricted via IAM
-

? What is bucket policy vs IAM policy?

Bucket Policy	IAM Policy
Applied to bucket	Applied to user/role

AWS Lambda – MISSED BASICS

? What is event source mapping?

Answer:

Configuration that connects Lambda to event sources like SQS or Kafka.

? What is DLQ (Dead Letter Queue)?

Answer:

Stores failed Lambda events for later processing.

? What is provisioned concurrency?

Answer:

Keeps Lambda warm to avoid cold start.

? Can Lambda run forever?

Answer:

No, max 15 minutes execution.

? What is stateless in Lambda?

Answer:

Each execution is independent (no shared memory).

RDS / AURORA – MISSED BASICS

? What is Aurora?

Answer:

A high-performance, cloud-optimized database compatible with MySQL/PostgreSQL.

? Difference: RDS vs Aurora?

RDS	Aurora
Standard DB	High performance
Lower cost	Higher cost but faster

RDS	Aurora
Single storage	Distributed storage

? What is backup in RDS?

Answer:

Automated snapshots + point-in-time recovery.

? What is failover?

Answer:

Switch to standby DB when primary fails.

? What is connection pooling?

Answer:

Reusing DB connections to improve performance.



LOAD BALANCER / ELB – MISSED BASICS

? Difference: ALB vs NLB?

ALB	NLB
Layer 7 (HTTP/HTTPS)	Layer 4 (TCP)
Smart routing	High performance

? What is target group?

Answer:

Group of EC2 instances where traffic is routed.

? What is listener?

Answer:

Defines port and protocol (e.g., HTTP:80).

? What is path-based routing?

Answer:

Routes traffic based on URL path

👉 /user → service A

👉 /order → service B

? What is cross-zone load balancing?

Answer:

Distributes traffic across all AZs evenly.

💣 BONUS BASIC QUESTIONS (INTERVIEW FAVORITES)

? How does request flow end-to-end?

✅ Answer:

Client → Load Balancer → EC2 → RDS / S3

? What is high availability?

Answer:

System continues working even if one component fails.

? What is scalability?

Answer:

Ability to handle increasing load.

? What is fault tolerance?

Answer:

System survives failures without downtime.

🎯 FINAL INTERVIEW TIP

If interviewer asks “**anything basic**”, they are actually testing:

- 👉 Clarity
- 👉 Fundamentals
- 👉 Real understanding

So always answer with:

- Definition
- Real use case
- One example

If you want to be fully unstoppable

I can give you:

- 🎯 ↳ 50 MOST ASKED AWS interview questions (company-wise)
- 💣 ↳ Rapid-fire round (like real interview pressure)
- 🧠 ↳ AWS + Spring Boot combined Q&A

Just tell me 👍

4/26/2026, 12:35:42 PM



advanced Q&A on s3, ec2,aws lambda, aurora db, loadbalancer ,
ELB, Amazon RDS

4/26/2026, 12:36:12 PM



Here are **advanced, interview-level Q&A** on

- 👉 Amazon EC2
- 👉 Amazon S3
- 👉 AWS Lambda
- 👉 Amazon RDS (Aurora included)
- 👉 Elastic Load Balancing

These are **real senior-level questions** where interviewers test architecture depth, trade-offs, and production thinking.



EC2 – ADVANCED Q&A

? How do you design fault-tolerant EC2 architecture?

✅ Answer:

- Deploy instances across multiple AZs
- Use Load Balancer
- Enable Auto Scaling
- Use health checks

👉 "This avoids single point of failure and ensures high availability."

? How do you handle memory leak in EC2 app?

✅ Answer:

- Monitor memory (CloudWatch)
- Restart instance (temporary fix)
- Fix code leak
- Add Auto Scaling as fallback

? When would you NOT use EC2?

✅ Answer:

- Event-driven workloads → use Lambda
 - Short-lived tasks → Lambda
 - Fully managed container → ECS/EKS
-

? How do you optimize EC2 cost?

✓ Answer:

- Use Reserved Instances
 - Use Spot Instances
 - Right-size instances
 - Stop unused instances
-

S3 – ADVANCED Q&A

? How do you design secure file upload system using S3?

✓ Answer:

- Use pre-signed URLs
 - Keep bucket private
 - Validate file before upload
 - Use IAM roles
-

? How do you handle large file uploads (GBs)?

✓ Answer:

- Use multipart upload
 - Upload directly from frontend to S3
 - Avoid backend bottleneck
-

? How do you ensure data consistency in S3?

✓ Answer:

“S3 provides strong read-after-write consistency for new objects.”

? How do you optimize S3 cost?

✓ Answer:

- Use lifecycle policies
 - Move to Glacier
 - Delete unused objects
-

? What is eventual consistency (older concept)?

✓ Answer:

Older S3 had eventual consistency for overwrite/delete, now mostly strong consistency.

⚡ AWS LAMBDA – ADVANCED Q&A

? How do you design high-throughput Lambda system?

✓ Answer:

- Use async triggers (SQS/Kafka)
 - Increase concurrency limits
 - Optimize execution time
-

? How do you manage DB connections in Lambda?

✓ Answer:

- Use connection pooling carefully
 - Reuse connections (outside handler)
 - Use RDS Proxy
-

? How do you handle cold starts in production?

✓ Answer:

- Provisioned concurrency
 - Reduce package size
 - Use lightweight runtime
-

? When Lambda is NOT suitable?

✓ Answer:

- Long-running tasks
 - Stateful apps
 - Heavy CPU workloads
-

? How do you ensure idempotency?

✓ Answer:

- Use unique request ID
 - Avoid duplicate processing
-

RDS / AURORA – ADVANCED Q&A

? How does Aurora improve performance over RDS?

✓ Answer:

- Distributed storage
 - Auto-scaling storage
 - Faster replication
 - Higher throughput
-

? How do you handle high DB load?

✓ Answer:

- Use read replicas
 - Add caching layer
 - Optimize queries
 - Scale instance
-

? What is RDS Proxy?

✓ Answer:

Manages DB connections efficiently (especially for Lambda).

? How do you design multi-region DB?

✓ Answer:

- Use Aurora Global Database
 - Replicate across regions
-

? What happens during failover?

✓ Answer:

- Primary DB fails
 - Standby promoted automatically
 - Minimal downtime
-



LOAD BALANCER / ELB – ADVANCED Q&A

? How does ALB route traffic internally?

✓ Answer:

- Receives request
 - Evaluates listener rules
 - Routes to target group
-

? How do you design zero-downtime deployment?

✓ Answer:

- Blue-green deployment
 - Use Load Balancer to switch traffic
-

? What is connection draining?

✓ Answer:

Allows in-flight requests to complete before stopping instance.

? How do you debug uneven traffic distribution?

✓ Answer:

- Check cross-zone balancing
- Check instance health
- Check sticky sessions

? How do you handle sudden traffic spikes?

✅ Answer:

- Auto Scaling
 - Load Balancer
 - Caching
-

💣 CROSS-SERVICE ADVANCED QUESTIONS (VERY IMPORTANT)

? Your system is slow. Where do you check?

✅ Answer:

1. Load Balancer
 2. EC2 CPU
 3. DB performance (RDS)
 4. S3 latency
 5. Network
-

? Design highly scalable architecture

✅ Answer:

- LB + EC2 Auto Scaling
 - RDS (Multi-AZ + replicas)
 - S3 for storage
 - Cache layer
 - Monitoring
-

? How do you ensure end-to-end reliability?

✅ Answer:

- Retry mechanism
 - Circuit breaker
 - Monitoring
 - Failover
-

? How do you reduce cost without impacting performance?

✅ Answer:

- Right-sizing
 - Caching
 - Lifecycle policies
 - Spot instances
-

🎯 FINAL INTERVIEW STRATEGY

When answering advanced questions:

👉 Always include:

- Trade-offs
 - Scaling strategy
 - Failure handling
 - Cost optimization
-

🚀 If you want ultimate mastery

I can:

- 🎯 ↳ Conduct **advanced mock interview (5+ years level)**
- 💣 ↳ Give **system design with cross-question grilling**
- 🧠 ↳ Provide **Spring Boot + AWS deep integration Q&A**

Just say: *"Start advanced mock interview"* 👍

4/26/2026, 12:36:15 PM



aws architecture explanation

Explain the AWS Shared Responsibility Model.

Difference between Security Groups and Network ACLs.

Difference between S3, EBS, and EFS.

How to design a highly available application (Multi-AZ).

What are Spot Instances and when to use them.

Explain IAM roles and policies.

What is a NAT Gateway and why use it?

Scenario: Cost optimization for a high-traffic website.

Scenario: Disaster recovery planning.

Difference between Auto Scaling and Load Balancing.

4/26/2026, 12:36:55 PM



Here's a **clear, interview-ready AWS architecture explanation + Q&A** for all your topics—focused on *how to explain confidently with real-world thinking*.



1. AWS Architecture (How to Explain)

👉 Typical architecture:

```
Client → Load Balancer → EC2 (App Layer)
                        → RDS (DB)
                        → S3 (Storage)
Private Subnet ↔ NAT Gateway ↔ Internet
```

- Public subnet → Load Balancer
- Private subnet → EC2 + DB
- Storage → S3
- Outbound internet → NAT Gateway



2. AWS Shared Responsibility Model

👉 Amazon Web Services divides responsibility:

AWS (Security OF the cloud)

- Data centers

- Hardware
- Network infrastructure

Customer (Security IN the cloud)

- OS patching
- Application security
- IAM access
- Data encryption

👉 Interview line:

"AWS secures the cloud, we secure what we deploy inside it."

🔥 3. Security Groups vs NACLs

Feature	Security Group	NACL
Level	Instance	Subnet
Type	Stateful	Stateless
Rules	Allow only	Allow & Deny

👉 Easy line:

"Security Groups = instance firewall, NACL = subnet firewall"

📦 4. S3 vs EBS vs EFS

Using

- 👉 Amazon S3
- 👉 Amazon EC2

Service	Type	Use Case
S3	Object	Images, files
EBS	Block	Disk for EC2
EFS	File	Shared storage

🌐 5. Highly Available Architecture (Multi-AZ)

👉 Design:

- Deploy EC2 in **multiple AZs**

- Use
 - 👉 Elastic Load Balancing
- Use
 - 👉 Amazon RDS Multi-AZ

👉 **Flow:**

- AZ1 fails → traffic goes to AZ2

👉 **Interview line:**

"No single point of failure across AZs."

⚡ 6. Spot Instances

👉 On

Amazon EC2

What?

Unused AWS capacity at low cost

When to use?

- Batch jobs
- Data processing
- Non-critical workloads

NOT for:

- Critical apps (can terminate anytime)
-

🔑 7. IAM Roles and Policies

👉 Using

AWS IAM

- **Role** → temporary permissions
- **Policy** → defines permissions

👉 Example:

EC2 role → access S3 without credentials

🌐 8. NAT Gateway

👉 What?

Allows private instances to access internet

Why needed?

- EC2 in private subnet cannot access internet directly

👉 Flow:

Private EC2 → NAT Gateway → Internet

9. Scenario: Cost Optimization (High Traffic Website)

Problem:

High AWS bill

Answer:

- Use Auto Scaling
- Use Spot + Reserved instances
- Use CDN (CloudFront)
- Optimize DB queries
- Use S3 lifecycle

👉 **Strong line:**

“Optimize compute, storage, and data transfer cost together.”

10. Scenario: Disaster Recovery

Types:

- Backup & restore
- Pilot light
- Warm standby
- Multi-region active-active

Best practice:

- Use RDS backups
- Store data in S3
- Multi-AZ deployment

👉 **Interview line:**

"RPO and RTO define DR strategy."

⚖️ 11. Auto Scaling vs Load Balancing

Feature	Auto Scaling	Load Balancer
Purpose	Add/remove instances	Distribute traffic
Works on	Capacity	Traffic

👉 **Best answer:**

"They work together—Auto Scaling adjusts capacity, Load Balancer distributes traffic."

🎯 FINAL INTERVIEW SUMMARY (USE THIS 🔥)



"In AWS architecture, I design systems using Load Balancer for traffic distribution, EC2 for compute, RDS for transactional data, and S3 for storage. I ensure high availability using Multi-AZ deployment, secure access using IAM, and optimize cost using scaling strategies and storage policies."

🚀 Want next level?

I can:

- 🧠 ↳ Draw **your RCM project architecture (interview-ready diagram)**
- 💣 ↳ Ask **system design + cross-questions**
- 🎯 ↳ Give **real whiteboard explanation practice**

4/26/2026, 12:36:57 PM

Just tell me 👍



Give me 50 most asked AWS interview questions company-wise

4/26/2026, 12:37:34 PM



Here's a **company-wise list of 50 MOST ASKED AWS interview questions** (based on patterns from companies like Amazon, TCS, Infosys, Accenture, Cognizant, Wipro, Capgemini).

This is curated for **5-8 years experience (your level)**.

AMAZON (High-level + Deep Architecture)

Focus: System design + scalability

1. Design scalable system using AWS
2. Explain AWS Shared Responsibility Model
3. How do you design fault-tolerant architecture?
4. How does Auto Scaling work internally?
5. Explain eventual consistency vs strong consistency
6. How do you handle distributed transactions?
7. How do you optimize cost in large systems?
8. How do you secure microservices in AWS?

 Expect deep follow-ups and cross-questions

TCS / INFOSYS / WIPRO (Service-based)

Focus: Basics + real-time usage

9. What is EC2 and how do you use it?
10. Difference between S3 and EBS
11. What is RDS and Multi-AZ?
12. What is Load Balancer?
13. What is IAM role?
14. What is VPC?
15. What is Security Group?
16. What is Auto Scaling?

👉 They test clarity, not depth

🟦 ACCENTURE / CAPGEMINI (Scenario-based)

🔥 Focus: Real-world problems

17. How do you handle high traffic in AWS?
 18. What if EC2 instance goes down?
 19. How do you secure S3 bucket?
 20. What is NAT Gateway and why use it?
 21. How do you monitor applications?
 22. How do you debug production issue?
-

🟪 COGNIZANT (Mixed + practical)

🔥 Focus: Real + theoretical

23. Difference between ALB and NLB
 24. What is health check in Load Balancer?
 25. What is Read Replica in RDS?
 26. What is S3 versioning?
 27. What is Lambda and when to use?
 28. What is CloudWatch?
-

🟠 PRODUCT COMPANIES (Advanced + tricky)

🔥 Focus: Deep understanding

29. Why is system slow even after scaling?
 30. How do you handle database bottleneck?
 31. Can S3 replace database?
 32. How do you ensure idempotency?
 33. How do you design event-driven system?
 34. How do you handle partial failures?
-

COMMON QUESTIONS (EVERY COMPANY)

MUST PREPARE

35. Difference: Auto Scaling vs Load Balancer
 36. What is high availability?
 37. What is fault tolerance?
 38. What is scalability?
 39. What is region and availability zone?
 40. What is IAM policy vs role?
-

SCENARIO-BASED QUESTIONS (VERY IMPORTANT)

41. Website down → how do you debug?
 42. High latency → what will you check?
 43. DB connection timeout → solution?
 44. S3 access denied → why?
 45. Load Balancer unhealthy → why?
-

ADVANCED SCENARIOS (TOP COMPANIES)

46. Design multi-region architecture
 47. Disaster recovery strategy
 48. Cost optimization strategy
 49. Secure healthcare/financial system
 50. Design microservices using AWS
-

HOW TO USE THIS LIST

👉 For each question:

- Know **definition**
 - Add **real example** (your RCM project)
 - Add **AWS service mapping**
-

Pro Tip (Game Changer)

If interviewer asks:

👉 "Tell me about AWS"

Say:



"I have worked with EC2 for compute, S3 for storage, and RDS (Aurora) for database. We used Load Balancer for high availability and Kafka for async processing in our microservices-based RCM system."

🔥 If you want ultimate preparation

I can:

- 🎯 ↳ Ask you these 50 questions one-by-one (mock interview)
- 💣 ↳ Give answers + counter-questions
- 🧠 ↳ Prepare company-specific answers (Amazon vs TCS style)

Just say: **"Start mock interview"** 👍

4/26/2026, 12:37:35 PM



answer all the 50 questions

4/26/2026, 12:38:08 PM



That's a big set—but exactly what you need. I'll give you **concise, interview-ready answers** for all 50 questions. Focus on *clarity + real-world mapping*.

AMAZON / ADVANCED (1-8)

1. Design scalable system using AWS

Use Load Balancer + Auto Scaling (EC2), DB (RDS), storage (S3), caching, and async processing. Ensure horizontal scaling and fault tolerance.

2. Shared Responsibility Model

AWS secures infrastructure; customer secures applications, data, and access.

3. Fault-tolerant architecture

Deploy across multiple AZs, use Load Balancer, Auto Scaling, and DB failover.

4. Auto Scaling internals

Monitors metrics → triggers scaling policies → launches/terminates instances.

5. Eventual vs strong consistency

- Strong → immediate consistency
 - Eventual → delay in sync (used in distributed systems)
-

6. Distributed transactions

Use Saga pattern or event-driven approach instead of ACID across services.

7. Cost optimization

Right-size resources, use reserved/spot instances, lifecycle policies.

8. Secure microservices

IAM roles, HTTPS, API Gateway, security groups, encryption.

SERVICE-BASED BASICS (9–16)

9. What is EC2?

Amazon EC2 → virtual server.

10. S3 vs EBS

Amazon S3 = object storage

EBS = block storage for EC2.

11. RDS & Multi-AZ

Amazon RDS → managed DB, Multi-AZ = failover support.

12. Load Balancer

Elastic Load Balancing distributes traffic.

13. IAM role

Temporary permission assigned to services.

14. VPC

Virtual private network in AWS.

15. Security Group

Instance-level firewall.

16. Auto Scaling

Automatically adds/removes EC2 instances.

SCENARIO-BASED (17-22)

17. Handle high traffic

Auto Scaling + Load Balancer + caching.

18. EC2 down

Traffic shifts via Load Balancer; replace instance.

19. Secure S3

Block public access + IAM policies + encryption.

20. NAT Gateway

Allows private instances to access internet.

21. Monitoring

Use CloudWatch for logs and metrics.

22. Debug production issue

Check LB → EC2 → logs → DB → external services.

PRACTICAL (23-28)

23. ALB vs NLB

ALB = HTTP routing, NLB = high-performance TCP.

24. Health check

Verifies instance availability before routing.

25. Read replica

Improves DB read performance.

26. S3 versioning

Stores multiple versions of objects.

27. Lambda

AWS Lambda runs code without servers.

28. CloudWatch

Monitoring + logs service.

PRODUCT COMPANY (29–34)

29. Slow after scaling

Likely DB bottleneck or network latency.

30. DB bottleneck

Use indexing, caching, read replicas.

31. Can S3 replace DB?

No, no query/transaction support.

32. Idempotency

Ensure repeated requests give same result.

33. Event-driven system

Use queues (Kafka/SQS) for async processing.

34. Partial failures

Use retries, circuit breaker, fallback.

COMMON (35–40)

35. Auto Scaling vs Load Balancer

Scaling vs traffic distribution.

36. High availability

System stays up despite failures.

37. Fault tolerance

System continues working after failure.

38. Scalability

Ability to handle increasing load.

39. Region vs AZ

Region = location, AZ = data center.

40. IAM policy vs role

Policy = permissions, Role = entity using permissions.

SCENARIOS (41–45)

41. Website down

Check LB, EC2, app logs, DB.

42. High latency

Check CPU, DB, network, caching.

43. DB timeout

Check connection, load, security group.

44. S3 access denied

Check IAM or bucket policy.

45. LB unhealthy

Check health endpoint, app status, port.



ADVANCED (46–50)

46. Multi-region architecture

Deploy app + DB replication across regions.

47. Disaster recovery

Backup, failover, multi-region strategy.

48. Cost optimization

Use reserved, spot, lifecycle policies.

49. Secure healthcare system

Encryption, IAM, restricted access, compliance.

50. Microservices on AWS

EC2 + LB + RDS + S3 + messaging (Kafka/SQS).



FINAL INTERVIEW TIP

Don't just answer—**connect to your project**:

👉 “In my RCM system, we used EC2 + Load Balancer for scaling, RDS for data, and S3 for storage...”

If you want to become interview-ready at top level

I can:

-  ↳ Conduct full **mock interview** (50 questions live)
-  ↳ Ask **deep follow-ups** like Amazon interviewer
-  ↳ Prepare **system design answer** (whiteboard level)

Just say: **"Start mock interview"** 

4/26/2026, 12:38:09 PM



Ask deep follow-ups like an Amazon interviewer and answer the question also

4/26/2026, 12:41:54 PM



Alright—now we'll step into **Amazon-style deep follow-ups**.

These are not surface questions—they'll **dig into your thinking**. I'll ask → then show **how you should answer**.

1. SCALING FOLLOW-UP

? Q: You said you use Amazon EC2 + Load Balancer.

What happens if traffic suddenly increases 10x in 2 minutes?

✅ Strong Answer:

"Auto Scaling based on CPU alone may react slowly. So I would:

- Use predictive or scheduled scaling for known spikes
- Use request count or ALB metrics instead of just CPU
- Keep a buffer of minimum instances
- Use caching and CDN to reduce backend load

This ensures the system handles sudden spikes without downtime."

👉 Interviewer checks: **proactive vs reactive scaling**

2. DATABASE BOTTLENECK FOLLOW-UP

? Q: Even after scaling EC2, system is still slow. Why?

✅ Strong Answer:

"Scaling compute doesn't help if the bottleneck is the database.

In that case:

- I would check Amazon RDS CPU and connections
- Optimize queries and indexing
- Add read replicas
- Introduce caching layer

The root cause is usually DB contention.”

👉 They check: **Do you understand system bottlenecks**

💣 3. S3 DESIGN FOLLOW-UP

? Q: Why not upload files through backend instead of directly to Amazon S3?

✅ Strong Answer:

“Uploading via backend creates unnecessary load and latency.

Instead:

- Use pre-signed URLs for direct upload
- Backend only generates URL

This reduces server load and improves scalability.”

👉 They check: **efficiency thinking**

💣 4. FAILURE HANDLING FOLLOW-UP

? Q: One microservice fails during claim processing. What happens?

✅ Strong Answer:

“I would handle it using:

- Retry mechanism
- Dead-letter queue
- Saga pattern for rollback

This ensures partial failures don’t break the system.”

👉 They check: **resilience design**

💣 5. LOAD BALANCER FOLLOW-UP

? Q: Elastic Load Balancing shows all instances healthy, but users still see errors. Why?

✅ Strong Answer:

“Health checks may not reflect real application health.

Possible issues:

- Health endpoint is too simple
- Downstream dependency failure (DB, API)

- Partial functionality failure

I would validate end-to-end flow, not just health checks.”

👉 They check: **real debugging mindset**

💣 6. SECURITY FOLLOW-UP

? Q: How do you ensure EC2 can access S3 securely without exposing keys?

✅ Strong Answer:

“I would use IAM roles attached to EC2 instead of hardcoding credentials.

This ensures secure and temporary access.”

👉 They check: **best practices awareness**

💣 7. COST FOLLOW-UP

? Q: Your AWS bill increased suddenly. First 3 things you check?

✅ Strong Answer:

“I would:

1. Check EC2 usage (unused instances)
2. Check data transfer costs
3. Check S3 storage growth

Then use cost analysis tools to identify anomalies.”

👉 They check: **ownership mindset**

💣 8. LAMBDA FOLLOW-UP

? Q: Why not replace EC2 with AWS Lambda?

✅ Strong Answer:

“Lambda is ideal for short-lived, event-driven tasks.

Our system requires long-running services and persistent connections, so EC2 is more suitable.”

👉 They check: **trade-off clarity**

💣 9. DATA CONSISTENCY FOLLOW-UP

? Q: How do you ensure consistency across microservices?

✅ **Strong Answer:**

"We use eventual consistency with event-driven architecture.

Critical operations are handled carefully, and retries ensure consistency over time."

👉 They check: **distributed systems knowledge**

💣 10. DISASTER RECOVERY FOLLOW-UP

? **Q: Region goes down. What's your plan?**

✅ **Strong Answer:**

"I would:

- Use multi-region deployment
- Replicate data (RDS / S3)
- Route traffic using DNS failover

This minimizes downtime and data loss."

👉 They check: **business continuity thinking**

💣 11. DEBUGGING FOLLOW-UP

? **Q: API latency increased suddenly. Where do you start?**

✅ **Strong Answer:**

"I follow layered debugging:

1. Load Balancer latency
2. EC2 CPU/memory
3. Application logs
4. DB query performance

This helps isolate the bottleneck quickly."

💣 12. TRICKY QUESTION

? **Q: Everything looks fine, but system still fails randomly. Why?**

✅ **Strong Answer:**

"Possible hidden issues:

- Race conditions
- Thread/memory leaks

- External dependency failures
- Network latency

I would use detailed logging and tracing to identify root cause."

AMAZON INTERVIEW SECRET (VERY IMPORTANT)

They are NOT testing:

 Definitions

They ARE testing:

-  Thinking
 -  Trade-offs
 -  Debugging approach
 -  Ownership
-

If you want next level (highly recommended)

I can:

-  ↳ Run a **real Amazon-style mock interview (30 min)**
-  ↳ Interrupt you with cross-questions like real interviewer
-  ↳ Give feedback on your answers

Just say: **"Start Amazon mock interview"** 

4/26/2026, 12:41:55 PM



Interrupt me with cross-questions like a real interviewer

4/26/2026, 12:42:36 PM



Alright—let's do this like a real Amazon-style interview.

I'll **ask** → **you answer** → **I interrupt with cross-questions**. No long prep time.

ROUND 1 – SYSTEM DESIGN + AWS

? Question 1:

You said you built an RCM system on AWS using EC2, RDS, S3, and a Load Balancer.

👉 **Design how your system handles 10x traffic spike.**

Explain step-by-step.

🚫 **Don't overthink. Start answering.**

(When you answer, I will interrupt like this 📌)

- "Why only CPU-based scaling?"
 - "What happens before scaling kicks in?"
 - "Where is your bottleneck?"
 - "What if database crashes?"
-

ROUND 2 – FAILURE SCENARIO

After you answer Q1, I'll hit you with:

? Question 2:

Your system is returning 503 errors from Load Balancer.

Everything looks healthy.

👉 What will you do?

ROUND 3 – DEEP DIVE (TRAP)

Then:

? Question 3:

Why not replace EC2 with Lambda?

RULES (IMPORTANT)

- Answer like in real interview (short + structured)
 - Don't write essay
 - Think aloud
 - I WILL interrupt mid-answer
-

 Start with **Question 1 now**.

4/26/2026, 12:42:40 PM